Capstone Plan — End-to-End DevOps Pipeline (Jenkins → EKS) under $30

Oct 18, 2026 · @kamal vasvani

Read this first

The whole project fits in about $20 of the $30 cap if the EKS cluster exists only while someone is actively working, and is destroyed every night. Everything else in this plan is built around that one rule.

What is actually in the repo. It contains the ShopNow app only: a React storefront (`frontend/`), a React admin panel (`admin/`), a Node.js/Express API (`backend/`, port 5000) that needs MongoDB, three Dockerfiles, and some docs. There is no Jenkinsfile, Terraform, Ansible or Kubernetes manifest. Your team writes all of those.

Fix these before Sprint 1 ends:

1. Fork the repo into the team GitHub account (`prassanaganesh`). GitHub webhooks need admin rights on the repo, which you do not have on `suhailm67-sys`.
2. The frontend and admin Dockerfiles build with `USER_NAME=aryan`, so the app expects to live under `/aryan`. Leaving it empty produces the URL `//api`, which the browser treats as a different host. Replace those lines with explicit build args (Sprint 1 shows the change).
3. The admin nginx config proxies `/admin/`, but the admin app calls `/api/...`. Add an `/api/` proxy block to `admin/nginx/default.conf`.
4. All three images use `node:18-alpine`, which is end-of-life. Move to `node:20-alpine` and `nginx:1.27-alpine` so the image scan is not full of findings.
5. The "AWS account id" in the brief (PrassanaGanesh) is a name. ECR image URLs need the 12-digit account ID; get it from the console's top-right menu.

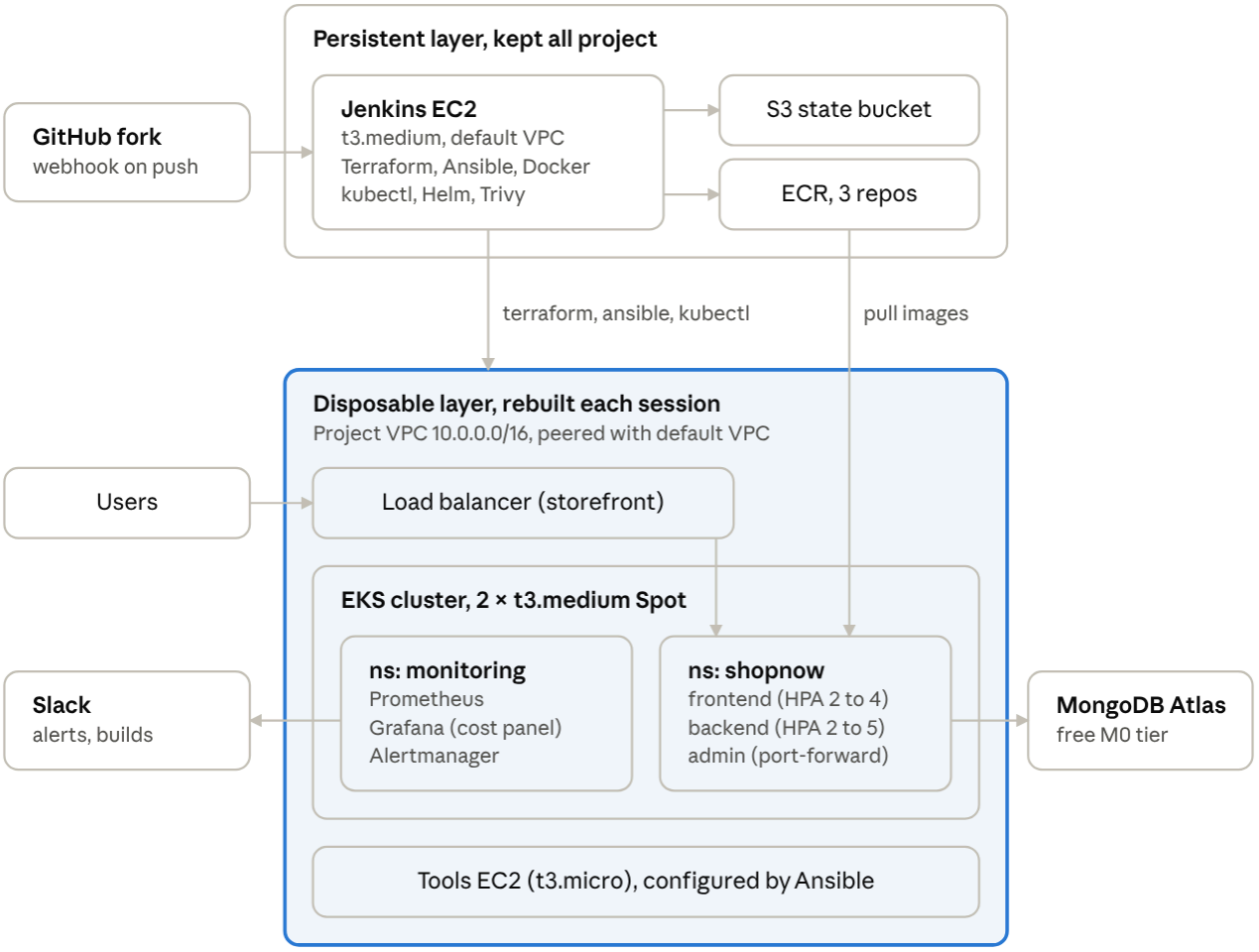
Scope mismatch to raise with your mentor. The brief's first deliverables list mentions a multi-cloud dashboard across AWS, Azure and GCP. The sprints only cover AWS, and "Multi-Cloud AWS Infrastructure" is contradictory. This plan covers AWS only, and meets the cost-monitoring ask with a Grafana cost panel plus AWS Budgets. Confirm that in writing before Sprint 2.

Account check. If this AWS account is on the Free account plan, confirm in Sprint 1 that EKS and Spot instances can be launched. Some services need the Paid plan, and new accounts sometimes have a Spot vCPU quota of 0.

Architecture and cost plan

Two layers keep the bill low. A small **persistent layer** (state bucket, ECR, a stoppable Jenkins EC2) lives for the whole project. A **disposable layer** (VPC, EKS, worker nodes, tools EC2, load balancer) is created by Jenkins at the start of a work session and destroyed at the end.

Only the persistent layer stays up; Jenkins rebuilds the rest each session



ShopNow on AWS · persistent and disposable layers

Everything inside the highlighted box is created by Jenkins at the start of a session and destroyed at the end; Jenkins, the state bucket and ECR persist.

Key design decisions, each one worth a line in the viva:

- **No NAT gateway.** Nodes run in public subnets with tight security groups. A NAT gateway alone would cost about $1.08/day plus data.
- **One load balancer, for the storefront only.** Admin, Grafana and Prometheus are reached with `kubectl port-forward`. Each extra load balancer adds about $0.60/day.
- **Spot worker nodes** (2 × t3.medium). Roughly a third of the on-demand price; on-demand is the fallback.

- **MongoDB Atlas free tier (M0)** for the database. Data survives the nightly cluster teardown and costs nothing. An in-cluster Mongo pod is the fallback, re-seeded each session.
- **Jenkins in the default VPC, peered to the project VPC.** Jenkins must exist before Terraform can build the project VPC. Peering is free and lets Prometheus scrape Jenkins privately.
- **EKS on a Kubernetes version in standard support.** Extended support raises the control-plane price from $0.10/hour to $0.60/hour.

Resources and approximate cost (us-east-1)

Prices are approximate list prices from memory; check the AWS pricing calculator once before Sprint 2.

Resource	Spec	Layer	Approx. price	Planned use	Approx. total
EKS control plane	1 cluster, standard support	Disposable	$0.10/hr	60 hrs	$6.00
EKS worker nodes	2 × t3.medium Spot, 20 GB gp3	Disposable	~$0.015/hr each	60 hrs	$2.00
Classic Load Balancer	Storefront Service	Disposable	~$0.025/hr	60 hrs	$1.50
Public IPv4 addresses	Nodes, tools EC2, LB, Jenkins	Both	$0.005/hr each	~5 IPs × 60 hrs + Jenkins	$2.00
Tools EC2	t3.micro, 8 GB gp3	Disposable	~$0.0104/hr	60 hrs	$0.70
Jenkins EC2	t3.medium, 20 GB gp3, stopped when idle	Persistent	~$0.042/hr + $1.60/month disk	120 hrs	$6.70
ECR	3 repos, keep last 5 images	Persistent	$0.10/GB-month	~2 GB	$0.40
S3 state bucket	Versioned, encrypted	Persistent	Pennies	—	$0.10
Data transfer, misc.	—	Both	—	—	$1.00
Total					**≈ $20**

The remaining ≈ $10 is the buffer for mistakes. A cluster forgotten overnight costs about $2.50; forgotten over a weekend, about $12. That is why the nightly auto-destroy job in Sprint 2 is mandatory, not optional.

Never create: NAT gateways, idle Elastic IPs, extra load balancers, EKS control-plane logging, a KMS key per cluster (the EKS module creates one by default; turn it off), or Persistent Volumes you forget to delete.

Team split and schedule

Each member owns one stream end to end, including its documentation chapter, so nobody waits on a single person.

Role	Owns	Docs chapter	Viva topic
M1 — Infra and cost lead	Step 0 guardrails, Terraform (bootstrap + platform), S3 state, destroy job, cost report	01-bootstrap, 03-terraform, 08-cost	VPC/EKS design, state locking, cost decisions
M2 — CI/CD lead	Jenkins server, plugins, credentials, webhook, all four Jenkinsfiles, notifications	02-jenkins	Pipeline stages, triggers, credentials and IAM role
M3 — App and Kubernetes lead	Dockerfile fixes, ECR, K8s manifests, probes, HPA, MongoDB, app tests	05-kubernetes, 07-testing	Deployment, health checks, autoscaling, rollback
M4 — Config and observability lead	Ansible playbooks, Prometheus, Grafana, alert rules, cost dashboard	04-ansible, 06-monitoring, 09-troubleshooting	Config management, metrics, alerting

Assign names to M1–M4 at your first meeting. Whoever has the strongest AWS and security background fits M1 best.

Schedule (adjust weeks to your deadline):

Week	Sprint	Done when	EKS hours allowed
1	Step 0 + Sprint 1	Budgets live, Jenkins up, images build locally and push to ECR from Jenkins	0

Week	Sprint	Done when	EKS hours allowed
2	Sprint 2	`01-infra-terraform` creates and destroys the full platform cleanly twice	8
3	Sprint 3	`02-config-ansible` runs automatically after Terraform and passes	10
4	Sprint 4	Git push → new pods on EKS without manual steps; HPA scales under load	14
5	Sprint 5	Dashboards populated; a test alert reaches Slack/email	12
6	Sprint 6	Two clean end-to-end runs recorded; docs and screenshots complete	12
		Total	**56**

No EKS in week 1 is deliberate: Docker images, ECR pushes and Jenkins setup need no cluster. Develop Kubernetes manifests locally first (Docker Desktop's built-in Kubernetes, kind or minikube) and only test on EKS once they work.

Repo layout

Add these folders to your fork; the existing `frontend/`, `admin/` and `backend/` stay where they are.

```
shopnow/  (your fork)
├── frontend/  admin/  backend/        # existing app code + Dockerfiles
├── Jenkinsfile                        # 03-shopnow-cicd (app pipeline)
├── jenkins/
|   ├── Jenkinsfile.infra              # 01-infra-terraform
|   ├── Jenkinsfile.config             # 02-config-ansible
|   └── Jenkinsfile.destroy            # 99-nightly-destroy
├── infra/
|   ├── bootstrap/                     # run once from CloudShell: Jenkins
EC2, IAM, ECR
|   └── platform/                      # run by Jenkins: VPC, EKS, tools EC2,
peering
├── ansible/
|   ├── ansible.cfg
|   ├── inventory/aws_ec2.yml          # dynamic inventory by tag
|   ├── site.yml
|   └── roles/  common  docker  cli_tools  node_exporter  k8s_addons
```

```
├── k8s/
│   ├── namespace.yaml
│   ├── backend.yaml  frontend.yaml  admin.yaml
│   ├── hpa.yaml
│   └── mongo-dev.yaml                # fallback only, if not using Atlas
├── monitoring/
│   ├── kps-values.yaml               # kube-prometheus-stack Helm values
│   ├── alert-rules.yaml
│   └── dashboards/
├── tests/
│   ├── smoke.sh
│   └── infra_check.sh
└── docs/   01-bootstrap.md … 09-troubleshooting.md
```

Branching: work on feature branches, merge to `main` through pull requests. Only `main` deploys to EKS; other branches run build and test stages only.

Never commit secrets. Add `*.tfstate*`, `.terraform/`, `*.pem` and `.env` to `.gitignore` on day one.

Step 0 — Guardrails and bootstrap (owner: M1, week 1)

Do this before any other AWS work. It takes about two hours and costs nothing until the Jenkins EC2 starts.

0.1 Account guardrails

1. Turn on MFA for the root user, then stop using root.
2. Create one IAM user per team member (or IAM Identity Center users), each with MFA. No shared logins; CloudTrail then shows who did what.
3. Billing → Budgets → create a monthly **cost budget of $30**. Add alerts at 33%, 66% and 85% of actual spend and 100% of forecast, emailed to all four members.
4. Billing → Billing preferences → enable **Receive CloudWatch billing alerts**. Grafana's cost panel reads this metric in Sprint 5.
5. Billing → Cost Anomaly Detection → create a monitor for AWS services with a daily email summary.
6. Use **us-east-1** for everything. Billing metrics only exist there.
7. EC2 → Key pairs → create `shopnow-key` (ED25519, .pem). Store it in a password manager, not in Git or chat.

0.2 State bucket (AWS CloudShell)

Use CloudShell from the console: it is free, already logged in, and avoids Windows tooling issues.

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
BUCKET=shopnow-tfstate-$ACCOUNT_ID
aws s3api create-bucket --bucket $BUCKET --region us-east-1
aws s3api put-bucket-versioning --bucket $BUCKET --versioning-configuration
Status=Enabled
aws s3api put-public-access-block --bucket $BUCKET --public-access-block-
configuration \

BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPub
licBuckets=true
aws s3api put-bucket-encryption --bucket $BUCKET --server-side-encryption-
configuration \
  '{"Rules":[{"ApplyServerSideEncryptionByDefault":
{"SSEAlgorithm":"AES256"}}]}'

# Terraform in CloudShell ($HOME persists between sessions)
mkdir -p ~/bin && cd ~/bin
curl -sLO
https://releases.hashicorp.com/terraform/1.13.4/terraform_1.13.4_linux_amd64.
zip
unzip -o terraform_*.zip && rm terraform_*.zip && terraform -version
```

Any Terraform 1.10 or newer works; it supports S3-native state locking (`use_lockfile`), so no DynamoDB table is needed.

0.3 Bootstrap Terraform (`infra/bootstrap/`)

Creates the persistent layer: ECR repos, the Jenkins IAM role and the Jenkins EC2.

```
# versions.tf
terraform {
  required_version = ">= 1.10"
  required_providers {
    aws  = { source = "hashicorp/aws", version = "~> 5.95" }
    http = { source = "hashicorp/http", version = "~> 3.4" }
  }
  backend "s3" {
    bucket       = "shopnow-tfstate-<ACCOUNT_ID>"
    key          = "bootstrap/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
```

```
    use_lockfile = true
  }
}
provider "aws" {
  region = "us-east-1"
  default_tags { tags = { Project = "shopnow", ManagedBy = "terraform" } }
}

# main.tf
variable "team_cidrs" { type = list(string) }   # each member's public IP as
x.x.x.x/32

data "aws_vpc" "default" { default = true }
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
}
data "http" "github_meta" { url = "https://api.github.com/meta" }
locals {
  github_hooks = [for c in
jsondecode(data.http.github_meta.response_body).hooks : c if !strcontains(c,
":")]
}

resource "aws_ecr_repository" "app" {
  for_each     = toset(["shopnow-frontend", "shopnow-admin", "shopnow-
backend"])
  name         = each.key
  force_delete = true
  image_scanning_configuration { scan_on_push = true }
}
resource "aws_ecr_lifecycle_policy" "keep5" {
  for_each   = aws_ecr_repository.app
  repository = each.value.name
  policy = jsonencode({ rules = [{ rulePriority = 1, description = "keep last
5",
    selection = { tagStatus = "any", countType = "imageCountMoreThan",
countNumber = 5 },
    action = { type = "expire" } }] })
}

resource "aws_iam_role" "jenkins" {
```

```
  name = "shopnow-jenkins-role"
  assume_role_policy = jsonencode({ Version = "2012-10-17", Statement = [{
    Effect = "Allow", Principal = { Service = "ec2.amazonaws.com" }, Action =
"sts:AssumeRole" }] })
}
resource "aws_iam_role_policy_attachment" "jenkins" {
  role       = aws_iam_role.jenkins.name
  policy_arn = "arn:aws:iam::aws:policy/AdministratorAccess"
}
resource "aws_iam_instance_profile" "jenkins" {
  name = "shopnow-jenkins-profile"
  role = aws_iam_role.jenkins.name
}

resource "aws_security_group" "jenkins" {
  name   = "shopnow-jenkins-sg"
  vpc_id = data.aws_vpc.default.id
  ingress {
    description = "SSH from team"
    from_port = 22
    to_port = 22
    protocol = "tcp"
    cidr_blocks = var.team_cidrs
  }
  ingress {
    description = "Jenkins UI from team + GitHub webhooks"
    from_port = 8080
    to_port = 8080
    protocol = "tcp"
    cidr_blocks = concat(var.team_cidrs, local.github_hooks)
  }
  ingress {
    description = "Prometheus scrape over VPC peering"
    from_port = 8080
    to_port = 9100
    protocol = "tcp"
    cidr_blocks = ["10.0.0.0/16"]
  }
  egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
resource "aws_instance" "jenkins" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.medium"
  key_name               = "shopnow-key"
  iam_instance_profile   = aws_iam_instance_profile.jenkins.name
  vpc_security_group_ids = [aws_security_group.jenkins.id]
  user_data              = file("${path.module}/jenkins-userdata.sh")
  root_block_device {
    volume_size = 20
    volume_type = "gp3"
  }
  metadata_options {
    http_tokens                 = "required"   # IMDSv2 only
    http_put_response_hop_limit = 2            # lets Docker builds reach the
instance role
  }
  tags = { Name = "shopnow-jenkins", Role = "jenkins" }
}

output "jenkins_url"         { value =
"http://${aws_instance.jenkins.public_ip}:8080" }
output "jenkins_instance_id" { value = aws_instance.jenkins.id }
output "ecr_repos"           { value = { for k, r in aws_ecr_repository.app :
k => r.repository_url } }
```

`jenkins-userdata.sh` installs only what Jenkins needs to start. Ansible installs everything else (Docker, kubectl, Helm, Terraform, Trivy) in Sprint 1, so Ansible is in use from week 1.

```
#!/bin/bash
set -eux
apt-get update
apt-get install -y fontconfig openjdk-21-jre-headless git unzip ansible
python3-boto3 python3-botocore
# Jenkins LTS apt repo: copy the current key + repo lines from
# https://www.jenkins.io/doc/book/installing/linux/#debianubuntu (the key
file name changes when rotated)
apt-get update && apt-get install -y jenkins
fallocate -l 2G /swapfile && chmod 600 /swapfile && mkswap /swapfile &&
swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab
systemctl enable --now jenkins
```

Run it from CloudShell:

```
cd infra/bootstrap
terraform init
terraform apply -var='team_cidrs=["1.2.3.4/32","5.6.7.8/32"]'
```

Why AdministratorAccess on the Jenkins role: the platform Terraform creates IAM roles for EKS, which narrower managed policies do not allow. Accept it only because this is a throwaway lab account, and compensate: Jenkins is reachable only from team IPs and GitHub, anonymous access is off, and each member has a named Jenkins login. Record this as an accepted risk in the docs.

0.4 First Jenkins login

1. `ssh -i shopnow-key.pem ubuntu@<ip>` → `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`.
2. Open the `jenkins_url` output, install suggested plugins, create one admin per member.
3. Manage Jenkins → Security: disable sign-up, use “Logged-in users can do anything” (or Matrix), and leave anonymous read off.
4. Stop the instance when you finish: `aws ec2 stop-instances --instance-ids <id>`. The public IP changes on each start; update the GitHub webhook URL afterwards (Sprint 1, step 1.5).

Sprint 1 — Architecture, Docker, Jenkins (week 1, no EKS)

Outcome: images for all three services build in Jenkins and land in ECR on every push to `main`. Spend: Jenkins hours only.

1.1 Architecture (all, owner M1)

Agree on the diagram in “Architecture and cost plan” above, then commit it as `docs/architecture.md` with the decision list. Add one line per decision explaining what the production version would use instead (private subnets + NAT, ALB ingress, managed MongoDB), so the viva can see you chose cost knowingly.

1.2 Fix and test the Dockerfiles (owner M3)

Frontend (`frontend/Dockerfile`); admin is identical apart from the comments:

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --no-audit
COPY . .
ARG PUBLIC_URL=/
ARG REACT_APP_API_BASE_URL=/api
ENV PUBLIC_URL=$PUBLIC_URL REACT_APP_API_BASE_URL=$REACT_APP_API_BASE_URL
RUN npm run build

FROM nginx:1.27-alpine
COPY nginx/default.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/build /usr/share/nginx/html
EXPOSE 80
HEALTHCHECK CMD wget -qO- http://127.0.0.1/health || exit 1
```

Add to `admin/nginx/default.conf`, beside the existing `/admin/` block:

```
location /api/ {
    proxy_pass       http://backend-service:5000/api/;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
```

Backend: change the base image to `node:20-alpine` and `npm ci --only=production` to `npm ci --omit=dev`. The health endpoint is `/api/health`.

The nginx configs proxy to a host called `backend-service`. Keep that exact name for the Kubernetes Service, and for the container name when testing locally:

```
docker network create shop
docker run -d --name mongo --network shop mongo:7
docker build -t shopnow-backend ./backend
docker run -d --name backend-service --network shop \
  -e MONGODB_URI=mongodb://mongo:27017/shopnow shopnow-backend
docker build -t shopnow-frontend ./frontend
docker run -d --name frontend -p 8081:80 --network shop shopnow-frontend
curl localhost:8081/health && curl localhost:8081/api/health
curl -X POST localhost:8081/api/seed/products     # loads sample products
```

1.3 MongoDB Atlas free cluster (owner M3)

1. Create a free M0 cluster on AWS us-east-1.
2. Create a database user with `readWrite` on database `shopnow` only and a long random password.

3. Network access: allow `0.0.0.0/0`. Worker node IPs change every session, so an allow list cannot be kept current; the strong password is the control. Note this in the docs.
4. Save the connection string (`mongodb+srv://…/shopnow`) as Jenkins credential `mongodb-uri`.

1.4 Configure the Jenkins host with Ansible (owner M4, with M2)

`ansible/jenkins-host.yml` installs the build tools. Run it over SSH once now and again whenever versions change.

```
- hosts: localhost
  connection: local
  become: true
  vars:
    k8s_minor: "1.33"        # match the EKS version you pick in Sprint 2
    terraform_version: "1.13.4"
  tasks:
    - name: Base packages
      ansible.builtin.apt:
        name: [docker.io, jq, curl, unzip, python3-kubernetes]
        state: present
        update_cache: true
    - name: Jenkins may use Docker
      ansible.builtin.user: { name: jenkins, groups: docker, append: true }
      notify: restart jenkins
    - name: AWS CLI v2
      ansible.builtin.shell: |
        curl -sSo /tmp/awscli.zip https://awscli.amazonaws.com/awscli-exe-
linux-x86_64.zip
        unzip -qo /tmp/awscli.zip -d /tmp && /tmp/aws/install --update
      args: { creates: /usr/local/bin/aws }
    - name: kubectl matching the cluster minor version
      ansible.builtin.shell: |
        V=$(curl -sL https://dl.k8s.io/release/stable-{{ k8s_minor }}.txt)
        curl -sLo /usr/local/bin/kubectl
https://dl.k8s.io/release/$V/bin/linux/amd64/kubectl
        chmod 0755 /usr/local/bin/kubectl
      args: { creates: /usr/local/bin/kubectl }
    - name: Helm 3
      ansible.builtin.shell: curl -fsSL
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
      args: { creates: /usr/local/bin/helm }
    - name: Terraform
      ansible.builtin.unarchive:
        src: "https://releases.hashicorp.com/terraform/{{ terraform_version
```

```
}}/terraform_{{ terraform_version }}_linux_amd64.zip"
        dest: /usr/local/bin
        remote_src: true
        creates: /usr/local/bin/terraform
    - name: Ansible collections for later sprints
      become_user: jenkins
      ansible.builtin.command: ansible-galaxy collection install amazon.aws
kubernetes.core community.general
  handlers:
    - name: restart jenkins
      ansible.builtin.service: { name: jenkins, state: restarted }
```

```
scp -i shopnow-key.pem -r ansible ubuntu@<ip>:~ && ssh -i shopnow-key.pem
ubuntu@<ip> \
  'sudo ansible-playbook ~/ansible/jenkins-host.yml'
```

Optional: install Trivy the same way for image scanning, pinned to a specific release rather than “latest”.

1.5 Plugins, credentials and Git trigger (owner M2)

Manage Jenkins → Plugins, install: Pipeline, Git, GitHub, Docker Pipeline, Kubernetes CLI, Pipeline: AWS Steps, AWS Credentials, Credentials Binding, Ansible, Prometheus metrics, Slack Notification (or Email Extension), Timestamper, Workspace Cleanup.

Credentials (Manage Jenkins → Credentials → global):

ID	Type	Used by
`github-pat`	Username + password (GitHub user + fine-grained PAT, contents read)	Checkout in every job
`ec2-ssh-key`	SSH username with private key (`ubuntu` / `ec2-user` , the `shopnow-key.pem`)	Ansible → tools EC2
`mongodb-uri`	Secret text	Kubernetes Secret for the backend
`slack-webhook`	Secret text (Slack incoming-webhook URL)	Notifications
`grafana-admin`	Secret text	Grafana admin password

No AWS access keys go into Jenkins. The AWS CLI, Terraform and kubectl pick up the `shopnow-jenkins-role` instance profile automatically; that is the "IAM roles" part of the sprint.

Git trigger:

1. GitHub fork → Settings → Webhooks → Add: payload URL `http://<jenkins-ip>:8080/github-webhook/` (trailing slash matters), content type `application/json`, push events only.
2. In the Jenkins pipeline job, tick "GitHub hook trigger for GITScm polling".
3. After each Jenkins start the IP changes. Update the hook from CloudShell instead of the UI:

```
IP=$(aws ec2 describe-instances --instance-ids <jenkins-id> \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text)
curl -s -X PATCH -H "Authorization: Bearer $GH_PAT" \
  https://api.github.com/repos/prassanaganesh/<fork-name>/hooks/<hook-id>/config \
  -d "{\"url\":\"http://$IP:8080/github-webhook/\",\"content_type\":\"json\"}"
```

This avoids an Elastic IP, which bills about $3.60/month while Jenkins is stopped.

1.6 First build-and-push job (owner M2)

Create pipeline job `03-shopnow-cicd` pointing at the root `Jenkinsfile`. For Sprint 1 it only checks out, builds the three images and pushes them; Sprint 4 adds test and deploy stages.

```
pipeline {
  agent any
  options { timestamps(); disableConcurrentBuilds() }
  environment {
    AWS_REGION = 'us-east-1'
    ACCOUNT_ID = sh(script: 'aws sts get-caller-identity --query Account --output text', returnStdout: true).trim()
    REGISTRY   = "${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com"
    TAG        = "${env.GIT_COMMIT.take(7)}-${env.BUILD_NUMBER}"
  }
  stages {
    stage('Build images') {
      steps {
        sh '''
          for svc in frontend admin backend; do
            docker build -t $REGISTRY/shopnow-$svc:$TAG ./$svc
```

```
          done
        '''
      }
    }
    stage('Push to ECR') {
      steps {
        sh '''
          aws ecr get-login-password --region $AWS_REGION | docker login --
username AWS --password-stdin $REGISTRY
          for svc in frontend admin backend; do docker push
$REGISTRY/shopnow-$svc:$TAG; done
        '''
      }
    }
  }
  post { always { sh 'docker image prune -af --filter "until=24h" || true' }
}
}
```

Sprint 1 is done when a push to `main` produces three new tags in ECR without anyone touching Jenkins.

Sprint 2 — Terraform platform via Jenkins (week 2, owner M1 + M2)

Outcome: one Jenkins click builds the VPC, EKS cluster, worker nodes and tools EC2 in about 15–20 minutes, and one click (or the nightly job) removes them.

2.1 Pick the Kubernetes version

```
aws eks describe-cluster-versions --region us-east-1 \
  --query 'clusterVersions[].
[clusterVersion,status,endOfStandardSupportDate]' --output table
```

Choose the newest version whose status is standard support. Put it in `cluster_version` and in `k8s_minor` in the Ansible vars; kubectl must be within one minor version of the cluster.

2.2 Platform Terraform (`infra/platform/`)

Same `versions.tf` as bootstrap, with backend key `platform/terraform.tfstate`.

```
# variables.tf
variable "env"             { default = "dev" }
variable "cluster_version" { default = "1.33" }     # from step 2.1
variable "node_capacity"   { default = "SPOT" }     # ON_DEMAND as fallback
variable "admin_principal_arns" {                   # team members' IAM user
ARNs
  type    = list(string)
  default = []
}
locals { name = "shopnow-${var.env}" }

# network.tf
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.21"

  name                   = local.name
  cidr                   = "10.0.0.0/16"
  azs                    = ["us-east-1a", "us-east-1b"]
  public_subnets         = ["10.0.1.0/24", "10.0.2.0/24"]
  enable_nat_gateway     = false            # biggest single saving
  map_public_ip_on_launch = true
  public_subnet_tags = {
    "kubernetes.io/role/elb"              = 1  # lets Kubernetes place the
load balancer
    "kubernetes.io/cluster/${local.name}" = "shared"
  }
}

# Peering so Jenkins (default VPC) reaches the tools EC2 and Prometheus
reaches Jenkins
data "aws_vpc" "default" { default = true }
data "aws_route_tables" "default" { vpc_id = data.aws_vpc.default.id }

resource "aws_vpc_peering_connection" "jenkins" {
  vpc_id      = module.vpc.vpc_id
  peer_vpc_id = data.aws_vpc.default.id
  auto_accept = true
  tags        = { Name = "${local.name}-to-default" }
}
resource "aws_route" "project_to_default" {
  route_table_id            = module.vpc.public_route_table_ids[0]
  destination_cidr_block    = data.aws_vpc.default.cidr_block
  vpc_peering_connection_id = aws_vpc_peering_connection.jenkins.id
}
resource "aws_route" "default_to_project" {
```

```
  for_each                  = toset(data.aws_route_tables.default.ids)
  route_table_id            = each.value
  destination_cidr_block    = "10.0.0.0/16"
  vpc_peering_connection_id = aws_vpc_peering_connection.jenkins.id
}

# eks.tf
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "~> 20.37"

  cluster_name                             = local.name
  cluster_version                          = var.cluster_version
  vpc_id                                   = module.vpc.vpc_id
  subnet_ids                               = module.vpc.public_subnets
  cluster_endpoint_public_access           = true
  authentication_mode                      = "API"
  enable_cluster_creator_admin_permissions = true   # the Jenkins role
becomes cluster admin

  # cost switches
  create_kms_key              = false
  cluster_encryption_config   = {}
  cluster_enabled_log_types   = []
  create_cloudwatch_log_group = false

  cluster_addons = {
    vpc-cni                = { before_compute = true }
    coredns                = {}
    kube-proxy             = {}
    eks-pod-identity-agent = {}
  }

  eks_managed_node_groups = {
    workers = {
      capacity_type              = var.node_capacity
      instance_types             = ["t3.medium", "t3a.medium"]
      min_size                   = 2
      max_size                   = 3
      desired_size               = 2
      use_custom_launch_template = false
      disk_size                  = 20
    }
  }

  access_entries = { for i, arn in var.admin_principal_arns : "member${i}" =>
```

```
{
    principal_arn = arn
    policy_associations = { admin = {
      policy_arn   = "arn:aws:eks::aws:cluster-access-
policy/AmazonEKSClusterAdminPolicy"
      access_scope = { type = "cluster" } } }
  } }
}

# tools.tf — the EC2 that Ansible configures in Sprint 3
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
}
resource "aws_security_group" "tools" {
  name   = "${local.name}-tools"
  vpc_id = module.vpc.vpc_id
  ingress {
    description = "SSH from Jenkins over peering only"
    from_port = 22
    to_port = 22
    protocol = "tcp"
    cidr_blocks = [data.aws_vpc.default.cidr_block]
  }
  ingress {
    description = "node_exporter from EKS nodes (Prometheus)"
    from_port = 9100
    to_port = 9100
    protocol = "tcp"
    security_groups = [module.eks.node_security_group_id]
  }
  egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
resource "aws_instance" "tools" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.micro"
  subnet_id              = module.vpc.public_subnets[0]
```

```
  key_name               = "shopnow-key"
  vpc_security_group_ids = [aws_security_group.tools.id]
  metadata_options { http_tokens = "required" }
  root_block_device {
    volume_size = 8
    volume_type = "gp3"
  }
  tags = { Name = "${local.name}-tools", Role = "tools", Env = var.env }
}

# grafana.tf — lets Grafana read the AWS billing metric (Sprint 5)
resource "aws_iam_role" "grafana" {
  name = "${local.name}-grafana"
  assume_role_policy = jsonencode({ Version = "2012-10-17", Statement = [{
Effect = "Allow",
    Principal = { Service = "pods.eks.amazonaws.com" }, Action =
["sts:AssumeRole", "sts:TagSession"] }] })
}
resource "aws_iam_role_policy_attachment" "grafana" {
  role       = aws_iam_role.grafana.name
  policy_arn = "arn:aws:iam::aws:policy/CloudWatchReadOnlyAccess"
}
resource "aws_eks_pod_identity_association" "grafana" {
  cluster_name    = module.eks.cluster_name
  namespace       = "monitoring"
  service_account = "kps-grafana"
  role_arn        = aws_iam_role.grafana.arn
}

# outputs.tf
output "cluster_name"     { value = module.eks.cluster_name }
output "tools_private_ip" { value = aws_instance.tools.private_ip }
```

Environments: keep `envs/dev.tfvars` and `envs/prod.tfvars` (different `env`, `node_capacity = "ON_DEMAND"` for prod). Only ever **apply** dev; run `plan` with prod for the viva to show the same code serves both.

Each env runs in its own Terraform workspace (selected in the Init stage), so dev and prod keep separate state files in the same bucket.

2.3 Jenkins job `01-infra-terraform` (`jenkins/Jenkinsfile.infra`)

```
pipeline {
  agent any
  parameters {
    choice(name: 'ACTION', choices: ['plan', 'apply', 'destroy'])
```

```
    choice(name: 'ENV', choices: ['dev', 'prod'])
    booleanParam(name: 'AUTO_APPROVE', defaultValue: false)
  }
  options { timestamps(); disableConcurrentBuilds(); timeout(time: 60, unit:
'MINUTES') }
  environment { TF_IN_AUTOMATION = 'true'; TF_INPUT = '0'; AWS_REGION = 'us-
east-1' }
  stages {
    stage('Init + validate') {
      steps { dir('infra/platform') {
        sh "terraform init -no-color && terraform workspace select -or-create
${params.ENV} && terraform fmt -check -recursive && terraform validate -no-
color"
      } }
    }
    stage('Plan') {
      when { expression { params.ACTION != 'destroy' } }
      steps { dir('infra/platform') { sh "terraform plan -no-color -var-
file=envs/${params.ENV}.tfvars -out=tfplan" } }
    }
    stage('Approve') {
      when { expression { params.ACTION != 'plan' && !params.AUTO_APPROVE } }
      steps { input message: "Run terraform ${params.ACTION} on
${params.ENV}?" }
    }
    stage('Apply') {
      when { expression { params.ACTION == 'apply' } }
      steps { dir('infra/platform') {
        sh 'terraform apply -no-color tfplan && terraform output -json >
../../tf-outputs.json'
      } }
    }
    stage('Destroy') {
      when { expression { params.ACTION == 'destroy' } }
      steps {
        sh 'bash tests/pre-destroy.sh || true'   // removes load balancers
first
        dir('infra/platform') { sh "terraform destroy -no-color -auto-approve
-var-file=envs/${params.ENV}.tfvars" }
      }
    }
  }
  post {
    success {
      archiveArtifacts artifacts: 'tf-outputs.json', allowEmptyArchive: true
      script { if (params.ACTION == 'apply') { build job: '02-config-
```

```
ansible', wait: false } }
    }
    failure { slackSend message: "01-infra-terraform ${params.ACTION} FAILED:
${env.BUILD_URL}" }
  }
}
```

`tests/pre-destroy.sh` deletes anything Kubernetes created in AWS, which Terraform does not know about and which blocks VPC deletion:

```
set -e
aws eks update-kubeconfig --name shopnow-dev --region us-east-1
kubectl get svc -A -o json | jq -r '.items[] |
select(.spec.type=="LoadBalancer") | "\(.metadata.namespace) \
(.metadata.name)"' |
  while read ns name; do kubectl delete svc -n "$ns" "$name"; done
kubectl delete pvc --all -A --wait=false || true
sleep 90   # give AWS time to remove the load balancer and its security group
```

2.4 Nightly safety net `99-nightly-destroy` (`jenkins/Jenkinsfile.destroy`)

```
pipeline {
  agent any
  triggers { cron('TZ=Asia/Kolkata\nH 23 * * *') }
  stages {
    stage('Destroy if cluster exists') {
      steps { script {
        def up = sh(script: 'aws eks describe-cluster --name shopnow-dev --
region us-east-1 >/dev/null 2>&1', returnStatus: true) == 0
        if (up) { build job: '01-infra-terraform', parameters: [string(name:
'ACTION', value: 'destroy'),
                  string(name: 'ENV', value: 'dev'), booleanParam(name:
'AUTO_APPROVE', value: true)] }
      } }
    }
    stage('Stop Jenkins itself') {
      steps { sh '''
        T=$(curl -sX PUT http://169.254.169.254/latest/api/token -H "X-aws-
ec2-metadata-token-ttl-seconds: 60")
        ID=$(curl -s -H "X-aws-ec2-metadata-token: $T"
http://169.254.169.254/latest/meta-data/instance-id)
        aws ec2 stop-instances --instance-ids $ID --region us-east-1
```

```
      ''' }
    }
  }
}
```

Team rule that makes this work: never stop the Jenkins EC2 by hand while the cluster exists. If Jenkins is off, the nightly job cannot run.

2.5 Test and done criteria

1. `ACTION=plan` with `ENV=prod`: plan succeeds, nothing created.
2. `ACTION=apply`, then `bash tests/infra_check.sh` (Sprint 6) passes.
3. Run `terraform plan -detailed-exitcode` again: exit code 0 means no drift, proving the build is reproducible.
4. `ACTION=destroy`, then confirm in the console: no cluster, no load balancers, no `available` EBS volumes.
5. Repeat apply and destroy once more. Two clean cycles completes the sprint (≈ 1.5 EKS hours each).

Sprint 3 — Configuration with Ansible via Jenkins (week 3, owner M4 + M2)

Outcome: when `01-infra-terraform` finishes an apply, `02-config-ansible` starts by itself, configures the tools EC2 and the cluster add-ons, and then triggers an app deploy.

Ansible configures two kinds of target. The **tools EC2** is configured over SSH. The **EKS worker nodes** are not: managed nodes are replaced rather than patched, so Ansible configures them through the Kubernetes API (namespaces, secrets, DaemonSets such as node-exporter, Helm add-ons). Say this explicitly in the viva; it answers the “configure Kubernetes nodes” requirement correctly.

3.1 Files

```
# ansible/ansible.cfg
[defaults]
inventory = inventory/aws_ec2.yml
roles_path = roles
host_key_checking = False
stdout_callback = yaml
[ssh_connection]
pipelining = True
```

```
# ansible/inventory/aws_ec2.yml — finds the tools EC2 by tag, no hard-coded
IPs
plugin: amazon.aws.aws_ec2
regions: [us-east-1]
filters:
  tag:Project: shopnow
  tag:Role: tools
  instance-state-name: running
hostnames: [private-ip-address]       # reached through VPC peering
keyed_groups:
  - key: tags.Role
    prefix: role                      # creates group role_tools
```

```
# ansible/site.yml
- name: Configure tools EC2
  hosts: role_tools
  gather_facts: false
  become: true
  pre_tasks:
    - ansible.builtin.wait_for_connection: { timeout: 300 }   # fresh
instance may still be booting
    - ansible.builtin.setup:
  roles: [common, docker, cli_tools, node_exporter]

- name: Configure EKS cluster add-ons
  hosts: localhost
  connection: local
  gather_facts: false
  roles: [k8s_addons]
```

3.2 Roles (keep each small)

Role	Does	Key tasks
`common`	Baseline OS hardening	apt upgrade, timezone Asia/Kolkata, `unattended-upgrades`, `PasswordAuthentication no` in sshd
`docker`	Container runtime	install `docker.io`, enable service, add `ubuntu` to `docker` group
`cli_tools`	Ops tooling	AWS CLI v2, kubectl (same tasks as `jenkins-host.yml`), `jq`

Role	Does	Key tasks
`node_exporter`	Host metrics for Prometheus	download a pinned release to `/usr/local/bin`, systemd unit on port 9100
`k8s_addons`	Cluster configuration	namespaces, Mongo secret, metrics-server, kube-prometheus-stack, alert rules

`roles/k8s_addons/tasks/main.yml`:

```
- name: Namespaces
  kubernetes.core.k8s:
    state: present
    definition: { apiVersion: v1, kind: Namespace, metadata: { name: "{{ item
}}" } }
  loop: [shopnow, monitoring]

- name: MongoDB connection secret (value comes from the Jenkins credential)
  kubernetes.core.k8s:
    state: present
    definition:
      apiVersion: v1
      kind: Secret
      metadata: { name: mongo, namespace: shopnow }
      type: Opaque
      stringData: { MONGODB_URI: "{{ lookup('env', 'MONGODB_URI') }}" }
  no_log: true

- name: Helm repos
  kubernetes.core.helm_repository: { name: "{{ item.name }}", repo_url: "{{
item.url }}" }
  loop:
    - { name: metrics-server, url: https://kubernetes-sigs.github.io/metrics-
server/ }
    - { name: prometheus-community, url: https://prometheus-
community.github.io/helm-charts }

- name: metrics-server (needed by the HPA)
  kubernetes.core.helm:
    name: metrics-server
    chart_ref: metrics-server/metrics-server
    release_namespace: kube-system
    wait: true

- name: kube-prometheus-stack (Sprint 5 fills in the values file)
  kubernetes.core.helm:
```

```
    name: kps
    chart_ref: prometheus-community/kube-prometheus-stack
    release_namespace: monitoring
    values_files: ["{{ playbook_dir }}/../monitoring/kps-values.yaml"]
    values:
      grafana: { adminPassword: "{{ lookup('env', 'GRAFANA_ADMIN_PASSWORD')
}}" }
    wait: true
    timeout: 15m0s

- name: Alert rules
  kubernetes.core.k8s:
    state: present
    src: "{{ playbook_dir }}/../monitoring/alert-rules.yaml"
```

Pin `chart_version` for both charts once the first install works, so reruns are repeatable.

3.3 Jenkins job `02-config-ansible` (`jenkins/Jenkinsfile.config`)

```
pipeline {
  agent any
  options { timestamps(); disableConcurrentBuilds(); timeout(time: 40, unit:
'MINUTES') }
  environment { AWS_REGION = 'us-east-1'; ANSIBLE_FORCE_COLOR = 'true' }
  stages {
    stage('Kubeconfig') {
      steps { sh 'aws eks update-kubeconfig --name shopnow-dev --region
$AWS_REGION && kubectl get nodes' }
    }
    stage('Syntax check') {
      steps { dir('ansible') { sh 'ansible-inventory --graph && ansible-
playbook site.yml --syntax-check' } }
    }
    stage('Configure') {
      steps {
        withCredentials([
          sshUserPrivateKey(credentialsId: 'ec2-ssh-key', keyFileVariable:
'SSH_KEY', usernameVariable: 'SSH_USER'),
          string(credentialsId: 'mongodb-uri', variable: 'MONGODB_URI'),
          string(credentialsId: 'grafana-admin', variable:
'GRAFANA_ADMIN_PASSWORD')]) {
          dir('ansible') {
            sh 'ansible-playbook site.yml --private-key "$SSH_KEY" -u
"$SSH_USER"'
            sh 'ansible role_tools -m command -a "docker --version" --
private-key "$SSH_KEY" -u "$SSH_USER"'
```

```
            }
          }
        }
      }
    }
    post {
      success { build job: '03-shopnow-cicd', parameters: [booleanParam(name:
'DEPLOY_ONLY', value: true)], wait: false }
      failure { slackSend message: "02-config-ansible FAILED: ${env.BUILD_URL}"
}
    }
}
```

The chain is now: `01-infra-terraform` (apply) → `02-config-ansible` → `03-shopnow-cicd` (deploy latest images). A second trigger is also fine: in `02-config-ansible` → Configure → "Build after other projects are built" → `01-infra-terraform`, but the `build job:` step above already handles it and only fires on apply.

3.4 Test and done criteria

1. Run `site.yml` twice on the same infrastructure. The second run should report `changed=0` (or nearly), which proves idempotency. Screenshot both recaps.
2. `ansible-playbook site.yml --check --diff` runs without errors.
3. On the tools EC2: `docker --version`, `kubectl version --client`, `curl -s localhost:9100/metrics | head` all succeed.
4. `kubectl get ns`, `kubectl -n monitoring get pods` and `kubectl top nodes` work.
5. A full apply in `01-infra-terraform` ends with the app deployed and nobody touching Jenkins in between.

Sprint 4 — CI/CD to EKS (week 4, owner M3 + M2)

Outcome: a push to `main` runs tests, builds and scans images, pushes to ECR, rolls out to EKS, smoke-tests through the load balancer, and rolls back on failure.

Write and test the manifests on a local cluster (Docker Desktop Kubernetes, kind or minikube) first. Use EKS only for the final runs.

4.1 Manifests (`k8s/`)

`__REGISTRY__` and `__TAG__` are placeholders the pipeline replaces with `sed`.

```
# k8s/backend.yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: backend, namespace: shopnow, labels: { app: backend } }
spec:
  # no replicas field: the HPA owns the count; a fixed value here would reset
it on every deploy
  selector: { matchLabels: { app: backend } }
  strategy: { type: RollingUpdate, rollingUpdate: { maxUnavailable: 0,
maxSurge: 1 } }
  template:
    metadata: { labels: { app: backend } }
    spec:
      containers:
        - name: backend
          image: __REGISTRY__/shopnow-backend:__TAG__
          ports: [{ name: http, containerPort: 5000 }]
          env:
            - name: MONGODB_URI
              valueFrom: { secretKeyRef: { name: mongo, key: MONGODB_URI } }
          resources:
            requests: { cpu: 100m, memory: 128Mi }
            limits:   { cpu: 500m, memory: 256Mi }
          readinessProbe:
            httpGet: { path: /api/health, port: http }
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet: { path: /api/health, port: http }
            initialDelaySeconds: 20
            periodSeconds: 15
            failureThreshold: 3
---
apiVersion: v1
kind: Service
metadata: { name: backend-service, namespace: shopnow, labels: { app: backend
} }   # name fixed by nginx config
spec:
  selector: { app: backend }
  ports: [{ name: http, port: 5000, targetPort: http }]
```

`k8s/frontend.yaml` is the same pattern with: image `shopnow-frontend`, container port 80, probes on `/health`, requests `50m` / `64Mi`, and a Service named `frontend` of `type: LoadBalancer` on port 80. This is the only public entry point.

`k8s/admin.yaml`: same as frontend, but its Service is `type: ClusterIP`. Reach it with `kubectl -n shopnow port-forward svc/admin 8082:80`, then open `http://localhost:8082`. Team laptops need the AWS CLI and kubectl for this (`winget install Amazon.AWSCLI Kubernetes.kubectl` on Windows) plus their IAM ARN in `admin_principal_arns`.

```
# k8s/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: backend, namespace: shopnow }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: backend }
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization:
60 } }
---
# same block for frontend: minReplicas 2, maxReplicas 4
```

Pod budget: t3.medium nodes hold at most 17 pods each (34 total). System pods use about 10, monitoring about 10, the app at full scale 10. It fits, but do not add replicas casually.

4.2 Pipeline `03-shopnow-cicd` (root `Jenkinsfile`)

```
pipeline {
  agent any
  parameters { booleanParam(name: 'DEPLOY_ONLY', defaultValue: false,
description: 'Skip build; redeploy newest ECR images') }
  triggers { githubPush() }
  options { timestamps(); disableConcurrentBuilds();
buildDiscarder(logRotator(numToKeepStr: '15')) }
  environment { AWS_REGION = 'us-east-1'; CLUSTER = 'shopnow-dev'; SERVICES =
'backend frontend admin' }

  stages {
    stage('Setup') { steps { script {
      env.ACCOUNT_ID = sh(script: 'aws sts get-caller-identity --query
Account --output text', returnStdout: true).trim()
      env.REGISTRY   =
"${env.ACCOUNT_ID}.dkr.ecr.${env.AWS_REGION}.amazonaws.com"
      env.TAG = params.DEPLOY_ONLY ?
        sh(script: "aws ecr describe-images --repository-name shopnow-backend
--query 'sort_by(imageDetails,&imagePushedAt)[-1].imageTags[0]' --output
```

```
text", returnStdout: true).trim() :
        "${env.GIT_COMMIT.take(7)}-${env.BUILD_NUMBER}"
      env.CLUSTER_UP = (sh(script: 'aws eks describe-cluster --name $CLUSTER
>/dev/null 2>&1', returnStatus: true) == 0).toString()
    } } }

    stage('Unit tests') {
      when { expression { !params.DEPLOY_ONLY } }
      parallel {
        stage('frontend') { steps { sh 'docker run --rm -u $(id -u):$(id -g)
-e HOME=/tmp -v "$PWD/frontend":/app -w /app node:20-alpine sh -c "npm ci --
no-audit && CI=true npm test -- --watchAll=false --passWithNoTests"' } }
        stage('admin')    { steps { sh 'docker run --rm -u $(id -u):$(id -g)
-e HOME=/tmp -v "$PWD/admin":/app -w /app node:20-alpine sh -c "npm ci --no-
audit && CI=true npm test -- --watchAll=false --passWithNoTests"' } }
        stage('backend')  { steps { sh 'docker run --rm -u $(id -u):$(id -g)
-e HOME=/tmp -v "$PWD/backend":/app -w /app node:20-alpine sh -c "npm ci --
no-audit && node --check server.js && npm run test --if-present"' } }
      }
    }

    stage('Build images') {
      when { expression { !params.DEPLOY_ONLY } }
      steps { sh 'for s in $SERVICES; do docker build -t
$REGISTRY/shopnow-$s:$TAG ./$s; done' }   // sequential: t3.medium memory
    }

    stage('Scan images') {
      when { expression { !params.DEPLOY_ONLY } }
      steps { sh 'for s in $SERVICES; do trivy image --severity HIGH,CRITICAL
--exit-code 0 --no-progress $REGISTRY/shopnow-$s:$TAG; done' }
      // switch to --exit-code 1 --severity CRITICAL once the base images are
updated
    }

    stage('Push to ECR') {
      when { expression { !params.DEPLOY_ONLY } }
      steps { sh '''
        aws ecr get-login-password --region $AWS_REGION | docker login --
username AWS --password-stdin $REGISTRY
        for s in $SERVICES; do docker push $REGISTRY/shopnow-$s:$TAG; done
      ''' }
    }

    stage('Deploy to EKS') {
      when { expression { env.CLUSTER_UP == 'true' } }
```

```
      steps { sh '''
        aws eks update-kubeconfig --name $CLUSTER --region $AWS_REGION
        for f in namespace backend frontend admin hpa; do
          sed -e "s|__REGISTRY__|$REGISTRY|g" -e "s|__TAG__|$TAG|g"
k8s/$f.yaml | kubectl apply -f -
        done
        for d in $SERVICES; do kubectl -n shopnow rollout status deploy/$d --
timeout=180s; done
      ''' }
    }

    stage('Smoke test') {
      when { expression { env.CLUSTER_UP == 'true' } }
      steps { sh '''
        LB=$(kubectl -n shopnow get svc frontend -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
        bash tests/smoke.sh "http://$LB"
      ''' }
    }
  }

  post {
    success { script {
      if (env.CLUSTER_UP != 'true') { currentBuild.result = 'UNSTABLE'; echo
'Cluster down: images pushed, deploy skipped' }
      slackSend color: 'good', message: "shopnow ${env.TAG} OK
(${currentBuild.currentResult}): ${env.BUILD_URL}"
    } }
    failure {
      sh 'if [ "$CLUSTER_UP" = true ]; then for d in $SERVICES; do kubectl -n
shopnow rollout undo deploy/$d || true; done; fi'
      slackSend color: 'danger', message: "shopnow ${env.TAG} FAILED, rolled
back: ${env.BUILD_URL}"
    }
    always { sh 'docker image prune -af --filter "until=24h" || true';
cleanWs() }
  }
}
```

Building while the cluster is down is intentional: developers can push code any time, and images land in ECR at no EKS cost. The build shows UNSTABLE as a reminder that nothing was deployed.

`tests/smoke.sh`:

```
#!/bin/bash
set -euo pipefail
URL=$1
for i in $(seq 1 30); do curl -sf "$URL/health" >/dev/null && break; sleep
10; done   # new LB DNS takes 2–3 min
curl -sf "$URL/health" | grep -q OK
curl -sf "$URL/api/health"
curl -sf "$URL/api/products" | jq -e . >/dev/null
echo "SMOKE TEST PASSED"
```

4.3 Test and done criteria

1. Change a visible string in `frontend/src/App.js`, push to `main`: the change is live at the LB URL within about 10 minutes, untouched by hand.
2. Seed data once per database: `curl -X POST http://<lb>/api/seed/products`.
3. **Autoscaling:** generate load and watch the HPA add pods.

```
kubectl -n shopnow run load --rm -it --image=busybox:1.36 --restart=Never --
\
  sh -c 'while true; do wget -q -O- http://backend-service:5000/api/products
>/dev/null; done'
kubectl -n shopnow get hpa -w      # in a second terminal
```

4. **Self-healing:** `kubectl -n shopnow delete pod -l app=backend` — replacements are Ready in under a minute and the site stays up.
5. **Rollback:** push a commit that breaks `/api/health`. Readiness fails, `rollout status` times out, the pipeline runs `rollout undo`, and the old pods keep serving throughout (`maxUnavailable: 0`). Record this; it is the strongest viva demo.

Sprint 5 — Prometheus, Grafana and alerts (week 5, owner M4)

Outcome: Grafana shows the app, the cluster, Jenkins and the AWS bill; Alertmanager posts to Slack when something breaks; Jenkins posts every build result.

All of it is installed by the `k8s_addons` Ansible role from Sprint 3, so it comes back automatically with every new cluster. Anything clicked together in the Grafana UI is lost at teardown (no persistent disk, by design), so keep dashboards as JSON files in Git.

5.1 Helm values as an Ansible template

Rename `monitoring/kps-values.yaml` to `kps-values.yaml.j2`. Ansible fills in IP addresses and secrets, so none are committed. Add a `template` task to `k8s_addons` that renders it to `/tmp/kps-values.yaml` and point the Helm task there.

```
grafana:
  persistence: { enabled: false }
  serviceAccount: { name: kps-grafana }        # matches the Pod Identity
association in Terraform
  additionalDataSources:
    - name: CloudWatch
      type: cloudwatch
      jsonData: { authType: default, defaultRegion: us-east-1 }
  sidecar: { dashboards: { enabled: true, searchNamespace: ALL } }

prometheus:
  prometheusSpec:
    retention: 2d
    resources: { requests: { cpu: 200m, memory: 512Mi }, limits: { memory:
1Gi } }
    serviceMonitorSelectorNilUsesHelmValues: false   # pick up our
ServiceMonitor
    ruleSelectorNilUsesHelmValues: false             # pick up our
PrometheusRule
    additionalScrapeConfigs:
      - job_name: jenkins
        metrics_path: /prometheus/
        static_configs: [{ targets: ["{{ jenkins_private_ip }}:8080"] }]
      - job_name: ec2-node-exporter
        static_configs:
          - targets: ["{{ jenkins_private_ip }}:9100", "{{
hostvars[groups['role_tools'][0]].private_ip_address }}:9100"]

alertmanager:
  config:
    route:
      receiver: slack
      group_by: [alertname, namespace]
      repeat_interval: 4h
      routes: [{ matchers: ['alertname = "Watchdog"'], receiver: "null" }]
    receivers:
      - name: "null"
      - name: slack
        slack_configs:
          - api_url: "{{ lookup('env', 'SLACK_WEBHOOK') }}"
            channel: "#shopnow-alerts"
            send_resolved: true
            title: '{{ "{{" }} .CommonLabels.alertname {{ "}}" }}'
            text: '{{ "{{" }} range .Alerts {{ "}}{{" }} .Annotations.summary
{{ "}}" }}\n{{ "{{" }} end {{ "}}" }}'
```

Set `jenkins_private_ip` with `lookup('pipe', "hostname -I | cut -d' ' -f1")`, and add `string(credentialsId: 'slack-webhook', variable: 'SLACK_WEBHOOK')` to the `withCredentials` block of `02-config-ansible`. Also add the `node_exporter` role to `jenkins-host.yml`, so both EC2s report host metrics.

5.2 Application metrics

The backend exposes no metrics yet. Add one package (two lines of code) so Prometheus can see request rate, errors and latency.

```
cd backend && npm install express-prom-bundle prom-client
```

```
// backend/server.js, directly after `const app = express();`
const promBundle = require('express-prom-bundle');
app.use(promBundle({ includeMethod: true, includeStatusCode: true,
promClient: { collectDefaultMetrics: {} } }));
```

```
# k8s/servicemonitor.yaml — add it to the pipeline's apply loop after hpa
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata: { name: backend, namespace: shopnow }
spec:
  selector: { matchLabels: { app: backend } }
  endpoints: [{ port: http, path: /metrics, interval: 30s }]
```

5.3 Alert rules (`monitoring/alert-rules.yaml`)

kube-prometheus-stack already ships many cluster alerts (crash loops, node pressure). These add the project-specific ones.

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata: { name: shopnow-alerts, namespace: monitoring }
spec:
  groups:
    - name: shopnow
      rules:
        - alert: BackendDown
          expr:
sum(kube_deployment_status_replicas_available{namespace="shopnow",deployment=
"backend"}) == 0
          for: 1m
          labels: { severity: critical }
          annotations: { summary: "ShopNow backend has no available pods" }
```

```
        - alert: HighErrorRate
          expr: |

sum(rate(http_request_duration_seconds_count{namespace="shopnow",status_code=
~"5.."}[5m]))
              /
sum(rate(http_request_duration_seconds_count{namespace="shopnow"}[5m])) >
0.05
          for: 5m
          labels: { severity: warning }
          annotations: { summary: "More than 5% of API requests are failing"
}
        - alert: NodeHighCPU
          expr: 100 - avg by (instance)
(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100 > 85
          for: 10m
          labels: { severity: warning }
          annotations: { summary: "{{ $labels.instance }} CPU above 85% for
10 minutes" }
        - alert: NodeLowMemory
          expr: node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes <
0.10
          for: 5m
          labels: { severity: warning }
          annotations: { summary: "{{ $labels.instance }} has under 10%
memory free" }
        - alert: JenkinsDown
          expr: up{job="jenkins"} == 0
          for: 2m
          labels: { severity: critical }
          annotations: { summary: "Prometheus cannot reach Jenkins" }
        - alert: JenkinsBuildFailed
          expr: increase(default_jenkins_builds_failed_build_count[15m]) > 0
          labels: { severity: warning }
          annotations: { summary: "Jenkins job {{ $labels.jenkins_job }}
failed" }
```

Check the exact Jenkins metric names at `http://<jenkins>:8080/prometheus/` before relying on the last rule; they differ slightly between plugin versions. Because this file is applied by Ansible, which also templates Jinja, wrap it in `{% raw %}` … `{% endraw %}` if you ever render it with `template` instead of `src`.

5.4 Dashboards (`monitoring/dashboards/`)

kube-prometheus-stack already includes cluster, node, namespace and pod dashboards. Add three as JSON files, each loaded by Ansible as a ConfigMap labelled `grafana_dashboard: "1"` so the Grafana sidecar picks it up:

Dashboard	Source	Panels that matter for the viva
Node Exporter Full	grafana.com ID 1860, download JSON	Jenkins and tools EC2 CPU, memory, disk
Jenkins	grafana.com, search “Jenkins Prometheus”, download JSON	Builds per job, failures, queue, executors
ShopNow (build it yourselves)	Export from Grafana UI after building	Request rate, p95 latency, 5xx %, pods per deployment, HPA replicas, AWS estimated charges

Useful queries for the ShopNow dashboard:

```
sum(rate(http_request_duration_seconds_count{namespace="shopnow"}[5m]))
# requests/s
histogram_quantile(0.95, sum by (le)
(rate(http_request_duration_seconds_bucket{namespace="shopnow"}[5m])))  # p95
latency
kube_horizontalpodautoscaler_status_current_replicas{namespace="shopnow"}
# HPA replicas
```

Cost panel: data source CloudWatch, namespace `AWS/Billing`, metric `EstimatedCharges`, dimension `Currency = USD`, statistic Maximum, period 6h. The metric updates a few times a day, so it shows the trend, not live spend.

Open Grafana with `kubectl -n monitoring port-forward svc/kps-grafana 3000:80` and Prometheus with `kubectl -n monitoring port-forward svc/kps-kube-prometheus-stack-prometheus 9090:9090`.

Historical data: Prometheus data lives only as long as the cluster (one session). Long-term history comes from AWS Cost Explorer and the CloudWatch billing metric, which persist. Say this plainly in the docs; it is a deliberate cost trade-off, and the production answer is Amazon Managed Prometheus or a persistent volume.

5.5 Jenkins notifications

The Jenkinsfiles call `slackSend`. Create a Slack app with the `chat:write` scope, invite it to `#shopnow-builds`, store its bot token as Jenkins credential `slack-bot-token`, then set Manage Jenkins → System → Slack: workspace, credential, default channel. Alertmanager uses the separate incoming webhook in `slack-webhook`. Email (Email Extension plugin + a Gmail app password) is an acceptable alternative if Slack is not allowed.

Cost alerts go through AWS Budgets and Cost Anomaly Detection from Step 0, which keep working while the cluster is down; that is the point.

5.6 Test and done criteria

1. Prometheus → Status → Targets: `jenkins`, `ec2-node-exporter`, `serviceMonitor/shopnow/backend` and the default targets are all UP.
2. `kubectl -n shopnow scale deploy backend --replicas=0` (the HPA pauses at zero replicas). `BackendDown` reaches Slack within about two minutes. Scale back to 2 and confirm the resolved message.
3. Push a commit that fails the unit tests: a Slack failure message from Jenkins and, within 15 minutes, `JenkinsBuildFailed`.
4. Run the load test from Sprint 4 with the ShopNow dashboard open; screenshot request rate and HPA replicas rising together.

Sprint 6 — Testing, documentation, final automation (week 6, all)

Outcome: two recorded end-to-end runs from an empty account state to a working, monitored app, plus documentation that lets a stranger repeat it.

6.1 Final trigger chain

After Sprints 2–5 the automation already chains. Confirm it end to end:

1. **Code push** → GitHub webhook → `03-shopnow-cicd` (test → build → scan → push → deploy → smoke test → notify).
2. **Infra start** → `01-infra-terraform` apply → `02-config-ansible` (configure + infra check) → `03-shopnow-cicd` with `DEPLOY_ONLY`.
3. **Every night 23:00 IST** → `99-nightly-destroy` → `01-infra-terraform` destroy → Jenkins stops itself.

6.2 Infrastructure check (`tests/infra_check.sh`)

Add it as the last stage of `02-config-ansible`.

```
#!/bin/bash
set -euo pipefail
R=us-east-1; C=shopnow-dev
ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
[ "$(aws eks describe-cluster --name $C --region $R --query cluster.status --
output text)" = ACTIVE ]
READY=$(kubectl get nodes --no-headers | grep -c ' Ready ')
[ "$READY" -ge 2 ] || { echo "only $READY nodes Ready"; exit 1; }
kubectl -n kube-system rollout status deploy/coredns --timeout=60s
kubectl top nodes >/dev/null                                    # metrics-
server works
aws s3api get-bucket-versioning --bucket shopnow-tfstate-$ACCOUNT | grep -q
Enabled
NAT=$(aws ec2 describe-nat-gateways --region $R --filter
Name=state,Values=available --query 'NatGateways[].NatGatewayId' --output
text)
[ -z "$NAT" ] || { echo "COST GUARD: NAT gateway found"; exit 1; }
echo "INFRA CHECK PASSED"
```

6.3 Test cases

Record each with date, result and a screenshot in `docs/07-testing.md`.

ID	Area	Test	Expected result
T01	App	Unit tests run in the pipeline	Stage green; a deliberately failing test turns the build red
T02	App	`tests/smoke.sh` against the LB URL	`/health`, `/api/health`, `/api/products` all return 200
T03	App	Place an order in the storefront, mark it ready in admin	Order appears in admin and its status changes
T04	Deploy	Push a UI text change to `main`	Live within ~10 minutes, no manual step
T05	Deploy	Push a commit that breaks `/api/health`	Rollout fails, automatic `rollout undo`, site stays up, Slack failure message
T06	Resilience	Delete all backend pods	Replaced and Ready in under 1 minute

ID	Area	Test	Expected result
T07	Scaling	Busybox load test	HPA scales backend from 2 to 3+ replicas, back down after load stops
T08	Infra	`01-infra-terraform` apply on empty state	`infra_check.sh` passes
T09	Infra	`terraform plan -detailed-exitcode` after apply	Exit code 0 (no drift)
T10	Infra	Destroy, then apply again	Same result; no leftover LBs, volumes or NAT gateways
T11	Config	Run `site.yml` twice	Second run `changed=0` or close to it
T12	Monitoring	All Prometheus targets	UP
T13	Alerting	Scale backend to 0	`BackendDown` in Slack within ~2 minutes, then resolved
T14	Cost	Budget alert	33% budget email received (or screenshot of the configured thresholds)

6.4 Documentation (15% of the grade)

One Markdown file per chapter in `docs/`, each owned by the member who built that part (see Team split). Each chapter uses the same five headings: Purpose, Prerequisites, Setup steps, Configuration reference, Troubleshooting.

File	Must include
`README.md`	Architecture diagram, one-paragraph overview, quick start (the daily routine), links to chapters
`01-bootstrap.md`	Guardrails, state bucket, bootstrap Terraform, accepted risks
`02-jenkins.md`	Plugins, credentials table (IDs only, never values), the four jobs and how they chain
`03-terraform.md`	Module diagram, variables, workspaces, state locking, how to destroy safely
`04-ansible.md`	Inventory, roles table, idempotency evidence

File	Must include
`05-kubernetes.md`	Manifests, probes, HPA, rollback behaviour, how to reach admin and Grafana
`06-monitoring.md`	Targets, dashboards, alert rules, notification channels
`07-testing.md`	The test table above with results and screenshots
`08-cost.md`	Cost table, every saving decision with its production alternative, **actual** Cost Explorer screenshot by service
`09-troubleshooting.md`	The troubleshooting table below, plus anything you hit yourselves

Screenshot checklist while the cluster is up (it saves EKS hours later): Jenkins pipeline stage view, ECR repos with tags, `kubectl get all -n shopnow`, HPA scaling, Grafana ShopNow and cost dashboards, Prometheus targets, Slack alerts, the storefront through the LB URL, Cost Explorer.

6.5 Final review

One member who did not build a part reviews it against its chapter by following the doc only. Whatever they get stuck on goes into the docs or the troubleshooting table. Then do the two final end-to-end runs and record one of them (screen capture) as a backup for the demo.

Daily work session routine

One person is the session owner each day and announces start and end in the team chat. Batch EKS work: four people testing together for three hours costs the same as one person for three hours.

Start (about 25 minutes, mostly waiting):

1. `aws ec2 start-instances --instance-ids <jenkins-id>`, then update the GitHub webhook with the CloudShell snippet from step 1.5.
2. Post in chat: “Cluster going up, owner: <name>”.
3. Jenkins → `01-infra-terraform` → ACTION `apply`, ENV `dev`. Config and app deploy follow automatically.
4. `aws eks update-kubeconfig --name shopnow-dev --region us-east-1` on each laptop that needs kubectl.

End (about 10 minutes):

1. Take any screenshots you still need.
2. Jenkins → `01-infra-terraform` → ACTION `destroy`.
3. Verify nothing is left:

```
aws eks list-clusters --region us-east-1
aws elb describe-load-balancers --query
'LoadBalancerDescriptions[].LoadBalancerName'
aws ec2 describe-volumes --filters Name=status,Values=available --query
'Volumes[].VolumeId'
aws ec2 describe-instances --filters Name=instance-state-name,Values=running
--query 'Reservations[].Instances[].Tags[?Key==`Name`].Value'
```

Expected: empty lists, except the Jenkins instance in the last one.

4. Stop Jenkins: `aws ec2 stop-instances --instance-ids <jenkins-id>`.
5. Post in chat: “Cluster down at HH:MM”.

Weekly (M1, 5 minutes): check Cost Explorer grouped by service and by usage type. Compare against the cost table; anything unexpected gets investigated the same day.

Troubleshooting

The problems most likely to cost you time or money, grouped by stage.

Symptom	Likely cause	Fix
`terraform destroy` hangs on subnets, IGW or VPC	A Kubernetes LoadBalancer or its security group still exists	Run `tests/pre-destroy.sh`; if already gone, delete the leftover CLB and `k8s-elb-*` security group in the console, then destroy again
`Error acquiring the state lock`	Another run in progress, or a crashed run left the lock	Wait for the other run. If none is running, delete the `.tflock` object next to the state file in S3
`UnsupportedAvailabilityZoneException` creating EKS	That AZ cannot host EKS for your account	Change `azs` to two other us-east-1 AZs

Symptom	Likely cause	Fix
Node group stuck creating, or no nodes	No Spot capacity, or Spot vCPU quota is 0 on a new account	Set `node_capacity = "ON_DEMAND"` (≈ $0.05/hr more), or request a Spot quota increase
`kubectl`: Unauthorized / must be logged in	Your IAM principal has no EKS access entry	Add your IAM user ARN to `admin_principal_arns` and re-apply; check `aws sts get-caller-identity` shows the expected identity
Pods Pending: `Too many pods`	t3.medium 17-pod limit per node	Reduce replicas or HPA max, or set node group `desired_size = 3` for that session
HPA targets show `<unknown>`	metrics-server missing, or containers have no CPU requests	`kubectl -n kube-system get deploy metrics-server`; confirm `resources.requests.cpu` on every container
`ImagePullBackOff`	Wrong tag or registry; image built for ARM on a Mac	Check the tag exists in ECR; build with `--platform linux/amd64` on Apple Silicon
Storefront loads, product list empty or 502	Backend not Ready, wrong Service name, or MongoDB unreachable	`kubectl -n shopnow logs deploy/backend`; Service must be named `backend-service`; check the `mongo` Secret and Atlas network access
Blank page, JS files 404	Image built with the old `USER_NAME` path prefix	Rebuild with the fixed Dockerfile (`PUBLIC_URL=/`)

Symptom	Likely cause	Fix
Jenkins: `permission denied ... docker.sock`	`jenkins` user not in `docker` group, or Jenkins not restarted	Re-run `jenkins-host.yml`; it adds the group and restarts Jenkins
ECR push: `no basic auth credentials`	Docker not logged in to ECR, or token expired (12 h)	`aws ecr get-login-password … | docker login …` runs in the push stage every time
Webhook shows red in GitHub, no build	Jenkins IP changed, or missing trailing slash	Re-run the webhook update snippet; URL must end `/github-webhook/`
Jenkins slow or killed during `npm run build`	Memory pressure on t3.medium	Confirm the 2 GB swap exists (`free -h`); build images one at a time (already the default)
Ansible: `UNREACHABLE` to tools EC2	Instance still booting, peering routes missing, or wrong SSH user	`wait_for_connection` covers boot; check routes in both VPCs and SG port 22 from 172.31.0.0/16; user is `ubuntu`
Prometheus target `jenkins` DOWN	Jenkins SG lacks 8080 from 10.0.0.0/16, or wrong metrics path	Check SG rule and peering routes; path is `/prometheus/` with trailing slash
Grafana CloudWatch panel: no data	Billing alerts not enabled, wrong region, or Pod Identity not applied	Enable billing alerts in Billing preferences; region us-east-1; `kubectl -n monitoring rollout restart deploy/kps-grafana`

Symptom	Likely cause	Fix
Bill higher than the cost table	Forgotten cluster, NAT gateway, idle Elastic IP, orphan EBS volumes or load balancer	Cost Explorer by usage type; run the end-of-session checks; destroy immediately